

Task 1 : Salary Transformation System:

```
import java.util.*;

import java.util.stream.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();

        int[] salaries = new int[N];

        for (int i = 0; i < N; i++) {

            salaries[i] = sc.nextInt();

        }

        int[] updatedSalaries = Arrays.stream(salaries)

            .map(salary -> salary + (salary * 10 / 100))

            .toArray();

        for (int i = 0; i < updatedSalaries.length; i++) {

            System.out.print(updatedSalaries[i]);

            if (i < updatedSalaries.length - 1) {

                System.out.print(" ");

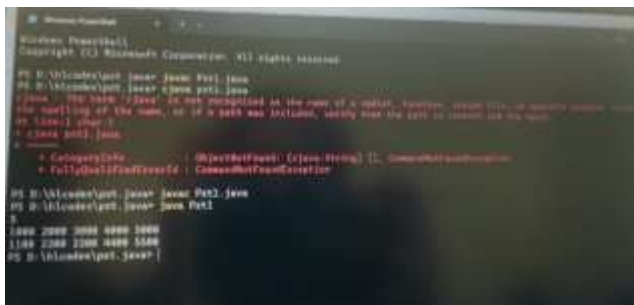
            }

        }

        sc.close();

    }

}
```



Task 2 : Real-Time Stream Analytics Engine

```
import java.util.*;

import java.util.stream.*;

class Reading {

    String sensorId;

    double temperature;

    Reading(String sensorId, double temperature) {

        this.sensorId = sensorId;

        this.temperature = temperature;

    }

}
```

```

    }
}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();

        List<Reading> readings = new ArrayList<>();

        for (int i = 0; i < N; i++) {

            String sensorId = sc.next();

            double temperature = sc.nextDouble();

            readings.add(new Reading(sensorId, temperature));

        }

        Map<String, Double> avgTemps =

            readings.stream()

                .filter(r -> r.temperature > 50)

                .collect(Collectors.groupingBy(

                    r -> r.sensorId,

                    Collectors.averagingDouble(r -> r.temperature)

                ));

        avgTemps.entrySet()

            .stream()

            .sorted((e1, e2) ->

                Double.compare(e2.getValue(), e1.getValue()))

            .forEach(e ->

                System.out.println(e.getKey() + " " + e.getValue()));

        sc.close();

    }

}

```



```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\kizodes\pat.java> java Pat2.java
5
s1 60
s2 60
s1 88
s3 70
s2 90
s3 30
s2 90.0
s3 70.0
s1 70.0
PS D:\kizodes\pat.java>

```

Task 3 : Maximum Profit Analyzer

```

import java.util.*;

public class Main {

    public static void main(String[] args) {

```

```

Scanner sc = new Scanner(System.in);

int N = sc.nextInt();

int[] arr = new int[N];

for (int i = 0; i < N; i++) {

    arr[i] = sc.nextInt();

}

int currentMax = arr[0];

int maxSoFar = arr[0];

for (int i = 1; i < N; i++) {

    currentMax = Math.max(arr[i], currentMax + arr[i]);

    maxSoFar = Math.max(maxSoFar, currentMax);

}

System.out.println(maxSoFar);

sc.close();

}
}

```

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\hlcodes\pst_java> java pst3.java
PS D:\hlcodes\pst_java> java pst3
error: Class names, 'pst3', are only accepted if annotation processing is explicitly requested
1 error
PS D:\hlcodes\pst_java> java pst3.java
PS D:\hlcodes\pst_java> java pst3
0
-2
-3
-5
-6
-7
-8
-4
-9
-2
PS D:\hlcodes\pst_java>

```

Task 4 : Intelligent DNA Pattern Search

```

import java.util.*;

public class Main {

    static void computeLPS(String pattern, int[] lps) {

        int len = 0;

        int i = 1;

        while (i < pattern.length()) {

            if (pattern.charAt(i) == pattern.charAt(len)) {

                len++;

                lps[i] = len;

                i++;

            } else {

                if (len != 0) {

                    len = lps[len - 1];

                } else {

                    lps[i] = 0;

                    i++;

                }

            }

        }

    }

}

```

```

    }
}
}
}

static void KMPSearch(String text, String pattern) {

    int n = text.length();
    int m = pattern.length();
    int[] lps = new int[m];
    computeLPS(pattern, lps);

    int i = 0;
    int j = 0;
    while (i < n) {
        if (pattern.charAt(j) == text.charAt(i)) {
            i++;
            j++;
        }
        if (j == m) {
            System.out.print((i - j) + " ");
            j = lps[j - 1];
        } else if (i < n && pattern.charAt(j) != text.charAt(i)) {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    String text = sc.nextLine();
    String pattern = sc.nextLine();
    KMPSearch(text, pattern);
    sc.close();
}
}

```

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\hlcodes\pst.java> javac pst4.java
PS D:\hlcodes\pst.java> java pst4
anabacandaabaaba
anba
0 9 12
PS D:\hlcodes\pst.java>

```

Task 5 : Banking Transaction System

```
import java.util.*;

class BankAccount {

    private int balance = 0;

    public void deposit(int amount) {

        balance += amount;

    }

    public void withdraw(int amount) {

        balance -= amount;

    }

    public int getBalance() {

        return balance;

    }

}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();

        BankAccount account = new BankAccount();

        for (int i = 0; i < N; i++) {

            String operation = sc.next();

            int amount = sc.nextInt();

            if (operation.equalsIgnoreCase("Deposit")) {

                account.deposit(amount);

            } else if (operation.equalsIgnoreCase("Withdraw")) {

                account.withdraw(amount);

            }

        }

        System.out.println(account.getBalance());

        sc.close();

    }

}
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\hlicodes\pst.java> javac task5.java
PS D:\hlicodes\pst.java> java task5.java
1
Deposit 5000 withdraw 2000 deposit 1000
4000
PS D:\hlicodes\pst.java> |
```